

LAB IMPLEMENTATION 9

ERC 20 Token Creation in Remix IDE

BC

Aim

To create an ERC 20 token using Solidity in Remix IDE and deploy the token contract to a selected testnet using a MetaMask wallet. The experiment also studies basic ERC 20 functions, events, and attributes.

Theory

An **ERC 20 token** is a fungible blockchain token implemented through a commonly used interface for token balances, transfers, and allowances. The `MyToken` contract defines public token attributes including `name`, `symbol`, `decimals`, and `totalSupply`, together with `balanceOf` and `allowance` mappings. The contract emits `Transfer` and `Approval` events when token balances or spending permissions change.

The contract provides `transfer` for sending tokens, `approve` for granting a spender an allowance, and `transferFrom` for transferring tokens using an approved allowance. Its constructor creates the initial supply and assigns it to the deploying account. Remix IDE is used to compile and deploy the contract, while MetaMask is used to approve transactions on the selected testnet. ERC 20 represents fungible tokens, whereas ERC 721 is intended for non-fungible tokens with individually distinguishable assets.

Steps

1. Open the LearnETH Solidity tutorials in Remix IDE and explore the ERC 20 Token course.
2. Study the six chapters provided in the ERC 20 Token course.
3. Set up a MetaMask wallet and select a supported testnet.
4. Fund the wallet with testnet cryptocurrency using an available faucet.
5. Create the Solidity file `MyToken.sol` and enter the ERC 20 token contract.
6. Compile `MyToken.sol` using a compatible Solidity compiler in Remix IDE.
7. Select the browser wallet deployment environment and connect MetaMask to the selected testnet.
8. Enter the initial token supply and approve the deployment transaction in MetaMask.
9. Deploy `MyToken` and inspect the deployed contract and transaction information.
10. Use the public token functions and attributes to inspect the token name, symbol, decimals, total supply, and balances.

Code

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.20;
3
4 contract MyToken {
5
6     string public name = "Jay Token";
7     string public symbol = "JAY";
8     uint8 public decimals = 18;
9
10    uint256 public totalSupply;
11
12    mapping(address => uint256) public balanceOf;
13
14    mapping(address => mapping(address => uint256)) public allowance;
15
16    event Transfer(
17        address indexed from,
18        address indexed to,
19        uint256 value
20    );
21
22    event Approval(
23        address indexed owner,
24        address indexed spender,
25        uint256 value
26    );
27
28    constructor(uint256 initialSupply) {
29        totalSupply = initialSupply * 10 ** decimals;
30        balanceOf[msg.sender] = totalSupply;
31
32        emit Transfer(address(0), msg.sender, totalSupply);
33    }
34
35    function transfer(
36        address to,
37        uint256 amount
38    ) public returns (bool) {
39
40        require(
41            balanceOf[msg.sender] >= amount,
42            "Insufficient balance"
43        );
44
45        balanceOf[msg.sender] -= amount;
46        balanceOf[to] += amount;
47
48        emit Transfer(msg.sender, to, amount);
49
50        return true;
51    }
52
53    function approve(
54        address spender,
55        uint256 amount
56    ) public returns (bool) {
57
```

```

58     allowance[msg.sender][spender] = amount;
59
60     emit Approval(msg.sender, spender, amount);
61
62     return true;
63 }
64
65 function transferFrom(
66     address from,
67     address to,
68     uint256 amount
69 ) public returns (bool) {
70
71     require(
72         balanceOf[from] >= amount,
73         "Insufficient balance"
74     );
75
76     require(
77         allowance[from][msg.sender] >= amount,
78         "Allowance exceeded"
79     );
80
81     balanceOf[from] -= amount;
82     balanceOf[to] += amount;
83
84     allowance[from][msg.sender] -= amount;
85
86     emit Transfer(from, to, amount);
87
88     return true;
89 }
90 }

```

Listing 1. MyToken.sol

Output

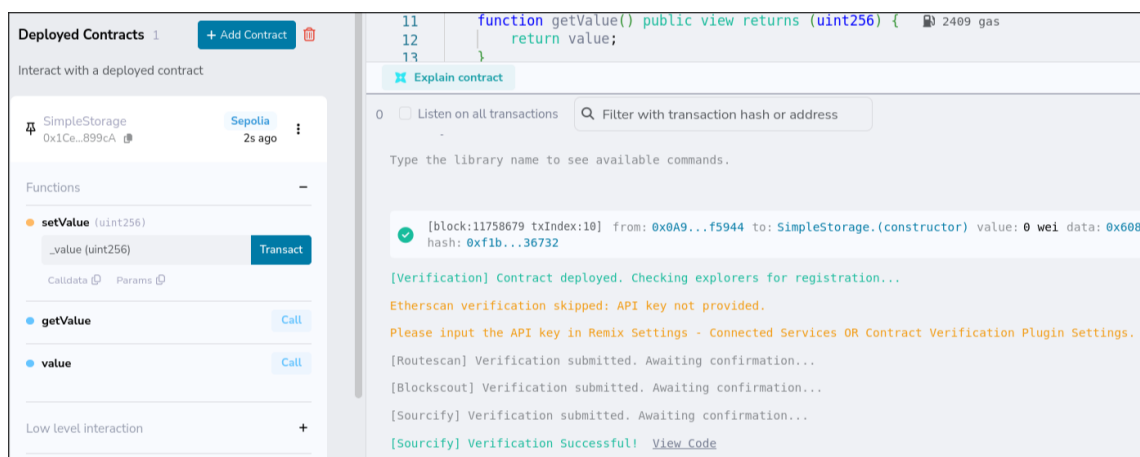


Fig. 1. MetaMask transaction request for deploying MyToken

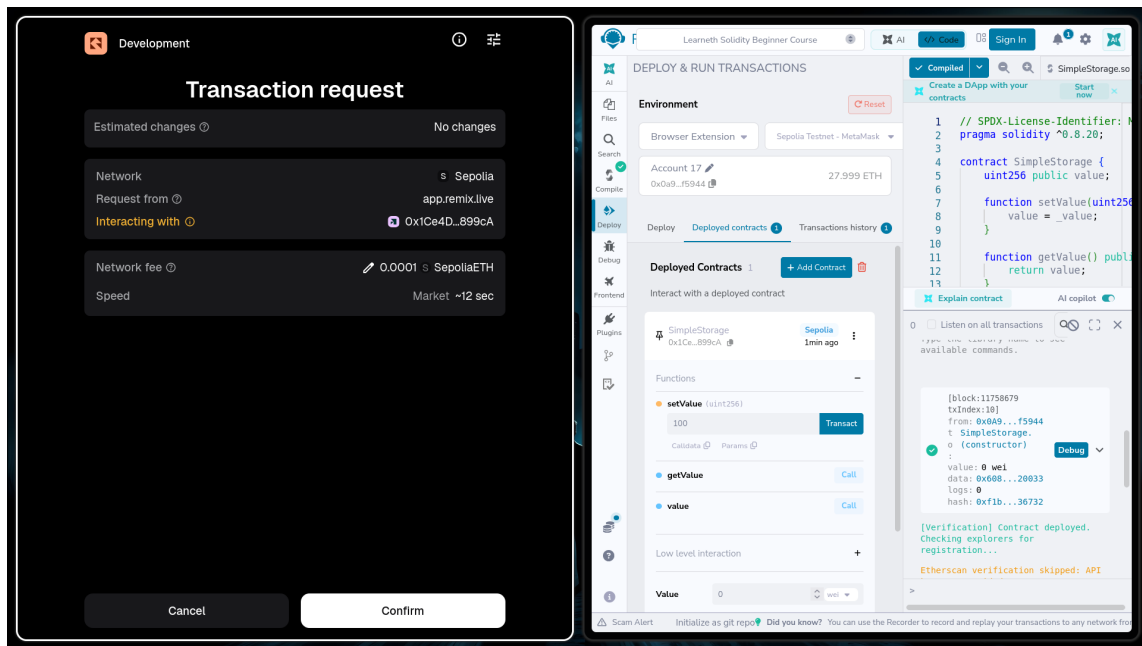


Fig. 2. MyToken deployment request and Remix deployment panel

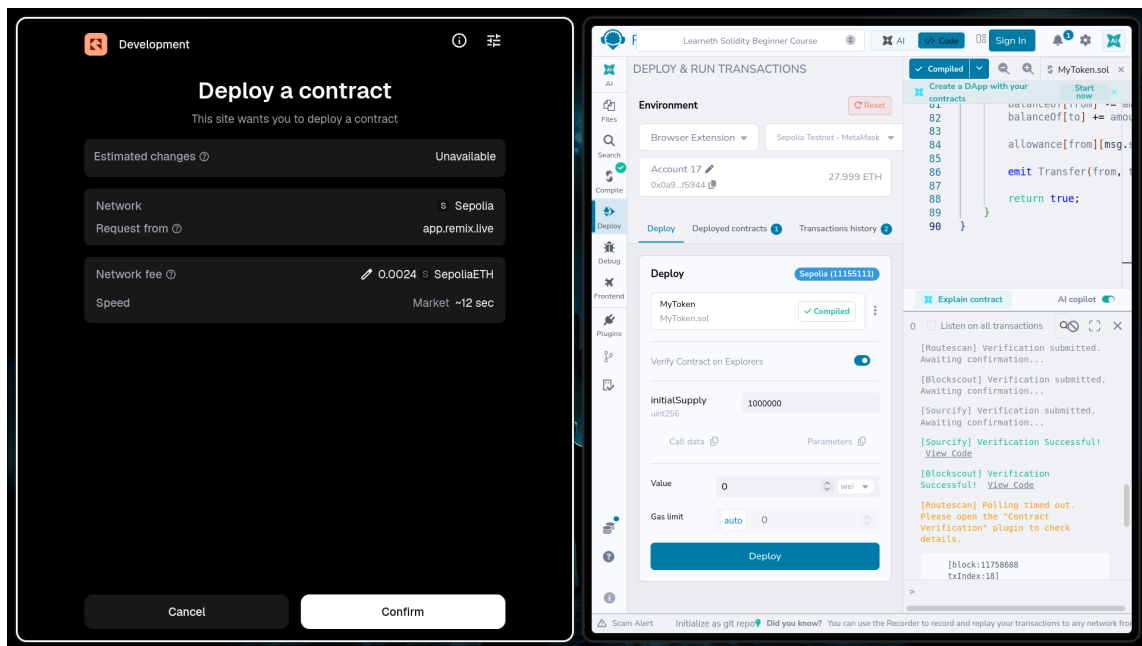


Fig. 3. Deployed MyToken contract with transaction details

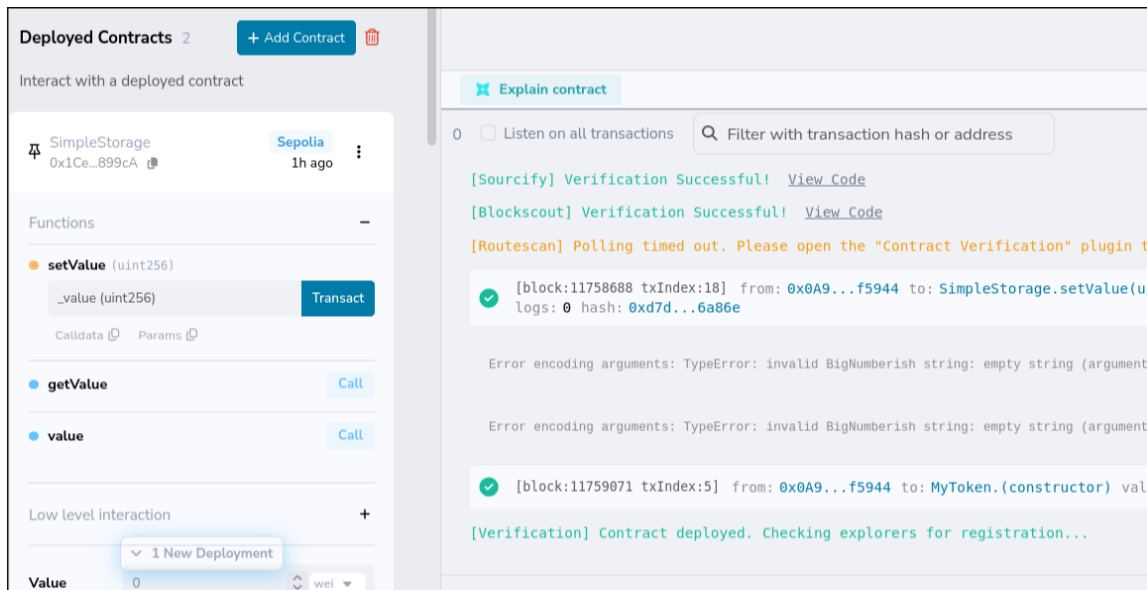


Fig. 4. MyToken compiler panel in Remix IDE

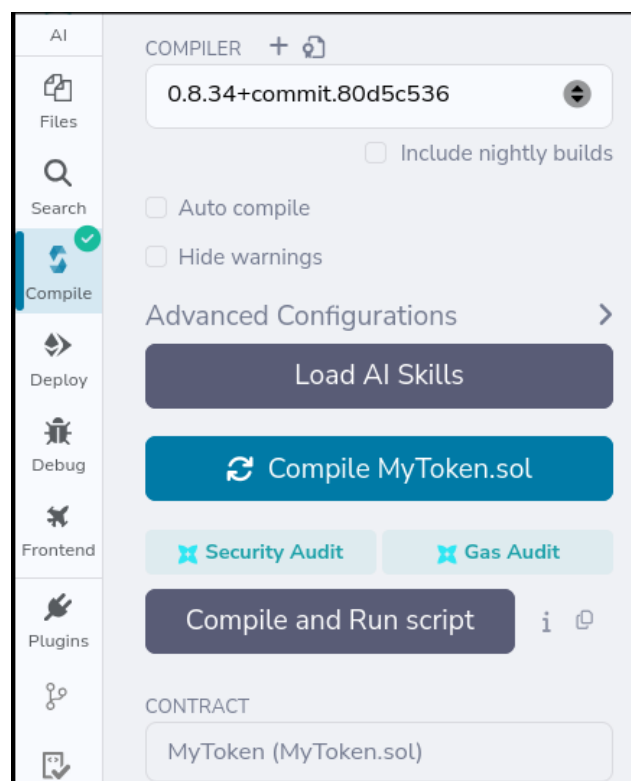


Fig. 5. MyToken public attributes and call results



Fig. 6. MyToken total supply call result

Conclusion

The MyToken Solidity contract was created with ERC 20-style token attributes, balance and allowance mappings, transfer and approval events, and transfer-related functions. The supplied output screenshots showed the MetaMask deployment transaction, MyToken deployment in Remix, deployed contract information, compilation, and public token attribute calls. The experiment demonstrated the process of creating and deploying a fungible token contract using Remix IDE and a MetaMask-connected testnet.